

AtCoder Library 移植模板

自包含可粘贴版·基于 [ac-library v1.6](#)

Build Time: August 5, 2026

Contents

1	AtCoder Library (ACL) 移植	2
1.1	segtree: 单点改 + 区间查 (非递归)	2
1.2	lazy_segtree: 区间改 + 区间查 (懒标记)	4
1.3	z_algorithm: Z 函数 (扩展 KMP)	8

1 AtCoder Library (ACL) 移植

本文档收录 [AtCoder Library v1.6](#) 里值得直接搬进赛场的组件，逐个改成自包含可粘贴版：

- 去掉 `#ifndef` 头尾守卫与 `namespace atcoder`，去掉 `std::` 前缀（模板里默认 `using namespace std;`）。
- 原本 `#include "atcoder/internal_bit"` 提供的 `bit_ceil / countr_zero` 内联成结构体私有 `static`，不再依赖任何 ACL 头文件。
- 模板参数统一用函数指针形式 `S (*op)(S, S)` 而非 C++17 的 `auto op`——写法与 C++14 兼容，且报错信息比 `auto + static_assert` 版本短得多。用 C++17 以上编译器时把 `S (*op)(S, S)`，`S (*e)()` 换成 `auto op`，`auto e` 也能编译，行为一致。

为什么要移植：ACL 只在 AtCoder 评测机上预装，ICPC / CCPC 赛场机没有 `atcoder/` 头文件目录，`#include <atcoder/segtree>` 直接 CE。本地开发（如 CMake 里 `include_directories(SYSTEM .../ac-library)`）可以照常用官方头，但打印稿上必须是能手抄进单文件的版本。

全文共同约定：区间一律 $[l, r]$ 左闭右闭，合法位置 $0 \dots n$ （构造 (n) 时内部开 $n+1$ 个叶子）。1-based 就只碰 $1 \dots n$ ，0-based 就只碰 $0 \dots n-1$ ，多出来的那格恒为 `e()` 不影响任何结果——两种习惯都不用换算，与主板子 `template-main` 各章一致。空区间写 $r = l - 1$ 。

这是本移植版对官方 ACL 的语义改动：官方 `atcoder/segtree` 用 $[l, r)$ 左闭右开、合法位置 $0 \dots n-1$ 。本文档在 `prod / apply / min_left` 入口处 `r++` 映射回内部的半开实现，`max_right` 取不到时返回 $l-1$ （官方返回 l ）。所以照着官方文档抄调用会差一位，以本文档的注释为准。

正确性验证：本文档代码块经 `template-check/` 下的 `acl_stress/run.sh` 与官方 ACL v1.6 及朴素暴力三方对拍（含 ASan / UBSan）通过。该脚本直接从本文档 `.tex` 抽 `minted` 块编译，改了代码不重跑就会暴露不一致。

1.1 segtree：单点改 + 区间查（非递归）

维护满足结合律的二元运算 `op` 与其单位元 `e`（即幺半群 `monoid`）上的序列，支持单点赋值 $O(\log n)$ 、区间求积 $O(\log n)$ 、以及在线段树上二分 $O(\log n)$ 。

接口一览（区间一律 $[l, r]$ 左闭右闭；`seg(n)` 的合法位置是 $0 \dots n$ ，共 $n+1$ 个叶子，所以 0-based 与 1-based 都能直接用，多出来的那一格恒为 `e()` 不影响结果）：

- `segtree<S, op, e> seg(n)`：合法位置 $0 \dots n$ ，全部初始化为 `e()`。
- `segtree<S, op, e> seg(v)`：由 `vector<S> v` 建树， $O(n)$ ；`v[i]` 就是位置 i ，长度自己定（1-based 用法写 `vector<S> a(n+1, e())` 再填 `a[1..n]`）。

- `seg.set(p, x)`：令 $a_p \leftarrow x$ 。
- `seg.get(p)`：返回 a_p 。
- `seg.prod(l, r)`：返回 $op(a_l, \dots, a_r)$ （闭区间）； $r = l - 1$ 表示空区间，返回 `e()`。
- `seg.all_prod()`：返回全区间积， $O(1)$ 。
- `seg.max_right(l, f)`：返回最大的 r ，使 $f(op(a_l, \dots, a_r))$ 为真（返回 $l-1$ 表示一个都取不到）。要求 f 单调（真在前假在后）且 $f(e())$ 为真。
- `seg.min_left(r, f)`：返回最小的 l ，使 $f(op(a_l, \dots, a_r))$ 为真（返回 $r+1$ 表示空）。同样要求 f 单调且 $f(e())$ 为真。

三个易错点：

- `op` 与 `e` 必须是自由函数（或函数指针 / 无捕获 `lambda` 转成的函数指针），不能是带捕获的 `lambda`——它们是模板参数，编译期就要定死。
- `e()` 必须是真单位元：求最大值时用足够小的哨兵（如 $-4e18$ ）而非 0，否则负数序列查询结果错。
- 从 `vector` 建树时长度不再自动 +1：想 1-based 用就得自己开 `a(n+1)`，直接传 `a(n)` 的话位置 n 不合法。另外 n 恰为 2 的幂时多开的那格会让 `bit_ceil` 翻倍（ $n = 2^{20} \Rightarrow$ d 数组 2^{22} ），卡空间的题留意。

示例一：区间最大值（单点赋值 + 区间查最大，完整可提交骨架）

```
/* 题意：n 个数，q 次操作
 * 1 p v -> 把第 p 个数改成 v (1-based)
 * 2 l r -> 输出 [l, r] 里的最大值 (闭区间)
 */
using S = ll; // 每个结点存什么：这里就是值本身

// op: 两段的答案怎么合并成一段的答案
// 必须满足结合律 —— max 满足：
// max(max(a, b), c) == max(a, max(b, c))
S op_max(S a, S b) { return max(a, b); }

// e: op 的单位元，要求 op(e(), x) == x 恒成立
// 所以取「不可能出现的极小值」当哨兵。
// 千万别写 0：全是负数时查询会错答成 0
S e_max() { return -4e18; }

int main() {
    int n, q;
    cin >> n >> q;

    // 想 1-based 用就自己开 n+1 格，a[i]
    // 就是位置 i
    vector<S> a(n+1, e_max());
    for (int i = 1; i <= n; i++) cin >> a[i];

    // 用 vector 一次性建树是 O(n);
    // 先 segtree(n) 再逐个 set 是 O(n log n),
```

```

// 能一次建好就不要循环 set
segtree<S, op_max, e_max> seg(a);

while (q--) {
    int t;
    cin >> t;
    if (t == 1) {          // 单点赋值
        int p;
        ll v;
        cin >> p >> v;
        seg.set(p, v);     // 1-based,
        // 题面给什么用什么
    } else {               // 区间查最大
        int l, r;
        cin >> l >> r;
        // prod 就是闭区间 [l, r], 不用换算
        cout << seg.prod(l, r) << "\n";
    }
}
}

```

示例二：区间和 + 线段树上二分。求「从 l 出发最长的前缀，使区间和不超过 x 」——这是 `max_right` 的典型用途，把「外层二分 + 内层查询」的 $O(\log^2 n)$ 压到 $O(\log n)$ ：

```

using S = ll;
// 求和版三件套：加法结合，单位元是 0
S op_sum(S a, S b) { return a + b; }
S e_sum() { return 0; }

int main() {
    int n;
    ll x;
    cin >> n >> x;
    vector<S> a(n + 1, e_sum()); // 1-based
    for (int i = 1; i <= n; i++) cin >> a[i];
    segtree<S, op_sum, e_sum> seg(a);

    int l = 1; // 从位置 l 出发

    // 谓词 f 可以是「带捕获的 lambda」——
    // 它是普通函数参数，不是模板参数，
    // 这点跟 op / e 的限制不同。
    // 要求 f 单调：真在前、假在后，且 f(e()) 真。
    // 这里 sum <= x 单调的前提是元素非负
    // (有负数时前缀和会上下摆，谓词不单调，
    // max_right 的返回值就没有意义了)
    int r = seg.max_right(l, [&](S sum) {
        return sum <= x;
    });

    // r 是「最大的位置」，使闭区间和
    // a[l] + a[l+1] + ... + a[r] <= x
    // 取不到任何元素时 r == l-1
    // 所以能取的元素个数 = r - l + 1
    cout << r - l + 1 << "\n";

    // 对称接口：min_left(r, f) 从右往左找
    // 最小的 l 使闭区间 a[l..r] 满足 f
    int l2 = seg.min_left(n, [&](S sum) {
        return sum <= x;
    });
}

```

```

cout << n - l2 + 1 << "\n"; // 后缀能取几个
}

```

模板本体：

```

/* AtCoder Library v1.6 atcoder/segtree
→ 自包含移植 + 闭区间
* 用法：segtree(n) 合法位置 0..n (开 n+1
→ 个叶子)，区间 [l, r] 左闭右闭
* 1-based 就只碰 1..n, 0-based 就只碰
→ 0..n-1,
* 多出来的那个位置恒为 e(), 不影响任何查询
* 从 vector 建：v[i] 对应位置 i, 长度自己定
* 1-based → vector<S> a(n + 1, e());
→ a[1..n] 填数据；segtree seg(a);
* 空区间写 r == l - 1 (prod 返回 e())
* 注意：n 恰为 2 的幂时多开的那个叶子会让
→ bit_ceil 翻倍，卡空间的题留意
*/

template<class S, S (*op)(S, S), S (*e)()>
struct segtree {
public:
    segtree() : segtree(0) {}

    // 合法位置 0..n, 内部开 n+1 个叶子
    explicit segtree(int n)
        : segtree(vector<S>(n + 1, e())) {}

    // v[i] 对应位置 i, 合法位置 0..v.size()-1
    explicit segtree(const vector<S> &v)
        : _n(int(v.size())) {
        size = (int) bit_ceil((unsigned) _n);
        log = countr_zero((unsigned) size);
        d = vector<S>(2 * size, e());
        for (int i = 0; i < _n; i++)
            d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--)
            update(i);
    }

    // a[p] = x (0 <= p <= n)
    void set(int p, S x) {
        assert(0 <= p && p < _n);
        p += size;
        d[p] = x;
        for (int i = 1; i <= log; i++)
            update(p >> i);
    }

    // 返回 a[p] (0 <= p <= n)
    S get(int p) const {
        assert(0 <= p && p < _n);
        return d[p + size];
    }

    // 返回 op(a[l], ..., a[r]) 闭区间
    S prod(int l, int r) const {
        r++;
        assert(0 <= l && l <= r && r <= _n);
        S sml = e(), smr = e();
        l += size, r += size;
        while (l < r) {

```

```

        if (l & 1) sml = op(sml, d[l++]);
        if (r & 1) smr = op(d[--r], smr);
        l >>= 1, r >>= 1;
    }
    return op(sml, smr);
}

S all_prod() const { return d[1]; }

// 最大的 r 使 f(op(a[l..r])) 为真 (闭区间);
// ↪ 取不到时返回 l-1
// 内部返回的是半开右端, 所以返回值统一 -1
template<class F>
int max_right(int l, F f) const {
    assert(0 <= l && l <= _n);
    assert(f(e()));
    if (l == _n) return _n - 1;
    l += size;
    S sm = e();
    do {
        while (l % 2 == 0) l >>= 1;
        if (!f(op(sm, d[l]))) {
            while (l < size) {
                l = 2 * l;
                if (f(op(sm, d[l]))) {
                    sm = op(sm, d[l]);
                    l++;
                }
            }
            return (l - size) - 1;
        }
        sm = op(sm, d[l]);
        l++;
    } while ((l & -l) != 1);
    return _n - 1;
}

// 最小的 l 使 f(op(a[l..r])) 为真 (闭区间);
// ↪ 取不到时返回 r+1
// 入参 r++ 成半开右端即可,
// ↪ 返回值本身就是闭区间左端
template<class F>
int min_left(int r, F f) const {
    r++;
    assert(0 <= r && r <= _n);
    assert(f(e()));
    if (r == 0) return 0;
    r += size;
    S sm = e();
    do {
        r--;
        while (r > 1 && (r % 2)) r >>= 1;
        if (!f(op(d[r], sm))) {
            while (r < size) {
                r = 2 * r + 1;
                if (f(op(d[r], sm))) {
                    sm = op(d[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(d[r], sm);
    } while ((r & -r) != r);
}

```

```

        return 0;
    }

private:
    int _n, size, log;
    vector<S> d;

    // 原 atcoder::internal::bit_ceil
    static unsigned bit_ceil(unsigned n) {
        unsigned x = 1;
        while (x < n) x *= 2;
        return x;
    }

    // 原 atcoder::internal::countr_zero
    static int countr_zero(unsigned n) {
        return __builtin_ctz(n);
    }

    void update(int k) {
        d[k] = op(d[2 * k], d[2 * k + 1]);
    }
};

```

与本模板其它线段树的取舍: ACL 版是非递归实现, 常数小、代码短, 但只支持单点修改; 要区间修改 (区间加 / 区间赋值) 请用下一小节的 lazy_segtree, 或主板子数据结构章的递归式懒标记线段树。ACL 版另一个优势是 max_right / min_left 这对树上二分接口, 主板子的线段树模板没有等价物。

1.2 lazy_segtree: 区间改 + 区间查 (懒标记)

在 segtree 的么半群 (S, op, e) 之上再挂一个作用 (映射) 么半群 $(F, composition, id)$, 支持区间作用 $O(\log n)$ 、区间求积 $O(\log n)$ 、树上二分 $O(\log n)$ 。

五件套的代数约束 (写错这里就是 WA 的主要来源):

- $S \text{ op}(S \ a, S \ b)$: 结合律, $S \ e()$ 是它的单位元。
- $S \text{ mapping}(F \ f, S \ x)$: 把作用 f 施加到 x 上。注意参数顺序是「先 f 后 x 」。
- $F \text{ composition}(F \ f, F \ g)$: 复合成「先 g 再 f 」, 即数学上的 $f \circ g$, 必须满足

$$\begin{aligned} \text{mapping}(\text{composition}(f, g), x) \\ = \text{mapping}(f, \text{mapping}(g, x)). \end{aligned}$$

- $F \text{ id}()$: F 的恒等作用, $\text{mapping}(\text{id}(), x) == x$ 。
- 分配律: 作用要对 op 分配, 即

$$\begin{aligned} \text{mapping}(f, \text{op}(x, y)) \\ = \text{op}(\text{mapping}(f, x), \text{mapping}(f, y)). \end{aligned}$$

这条不成立就不能用懒标记。

最大的坑：mapping 拿不到区间长度。做「区间加 + 区间和」时， $mapping(+v, x) = x + v \cdot len$ 需要 len ，而 $mapping$ 的入参只有 f 和 x ——所以长度必须存进 S 里，让它随 op 一起合并。这与主板子数据结构章那种「递归下传时天然知道 $[l, r]$ 」的写法不同，是 ACL 版最容易踩空的地方。

接口一览（比 `segtree` 多出 `apply`，且因为要 `push` 懒标记，`get / prod` 不再是 `const`）：

- `seg.apply(p, f)`：单点作用 $a_p \leftarrow mapping(f, a_p)$ 。
- `seg.apply(l, r, f)`：闭区间 $[l, r]$ 上每个元素都作用 f ； $r = l - 1$ 表示空，直接返回。
- `seg.set / get / prod / all_prod / max_right / min_left`：语义同 `segtree`，同样是闭区间、合法位置 $0 \dots n$ 。

示例一：区间加 + 区间和（洛谷 P3372 那类板子题，完整可提交骨架）

```
/* 题意：n 个数，q 次操作
 * 1 l r v -> 把 [l, r] 每个数加上 v (闭区间)
 * 2 l r -> 输出 [l, r] 的区间和
 */

// 结点存什么：和 + 长度。
// 为什么要存长度：mapping(f, x) 只拿到 f 和 x，
// 拿不到「这段有多长」，而区间加 v 让和增加
// v * 长度 —— 长度只能自己存进 S 里跟着 op 合并
struct S {
    ll sum; // 这段区间的元素和
    int len; // 这段区间的元素个数
};

using F = ll; // 懒标记：这段区间「待加上的数」

// op: 合并两段 —— 和相加，长度相加
S op(S a, S b) {
    return {a.sum + b.sum, a.len + b.len};
}

// e: op 的单位元 = 空区间 (和 0, 长度 0)
S e() { return {0, 0}; }

// mapping: 把「每个元素 +f」作用到整段 x 上
// 和增加 f * 元素个数；长度不变
S mapping(F f, S x) {
    return {x.sum + f * x.len, x.len};
}

// composition: 复合成「先 g 再 f」
// 先加 g 再加 f，等价于一次加 (f + g)
F composition(F f, F g) { return f + g; }

// id: 恒等作用 = 加 0 (表示
// 「这段没有待下传的加法」)
F id() { return 0; }

// 七个模板参数顺序：S, op, e, F,
// mapping, composition, id
using Seg = lazy_segtree<S, op, e, F,
    mapping, composition, id>;
```

```
mapping, composition, id>;

int main() {
    int n, q;
    cin >> n >> q;

    // 想 1-based 用就自己开 n+1 格，a[i]
    // 就是位置 i
    // a[0] 留 e() = {0, 0}，长度 0
    // 不影响任何区间和
    vector<S> a(n + 1, e());
    for (int i = 1; i <= n; i++) {
        ll x;
        cin >> x;
        a[i] = {x, 1}; // 单点：和 = x，
        // 长度 = 1
    }
    Seg seg(a); // 建树 O(n)

    while (q--) {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 1) { // 区间加
            ll v;
            cin >> v;
            // apply 就是闭区间 [l, r]，不用换算
            seg.apply(l, r, v);
        } else { // 区间求和
            // prod 返回的是 S，取 .sum
            cout << seg.prod(l, r).sum
                << "\n";
        }
    }
}
```

示例二：区间赋值 + 区间最大。赋值标记要能表达「这段没有被赋值」，用一个数据里不可能出现的值当哨兵：

```
/* 题意：1 l r v -> [l, r] 全部赋值为 v
 * 2 l r -> 输出 [l, r] 的最大值
 */

using S = ll; // 结点存这段的最大值
using F = ll; // 懒标记：这段待赋值的值

// 哨兵：表示「这段没有待下传的赋值」。
// 必须取一个数据里绝不会出现值，否则
// 「真的赋值成 NONE」会被误判成「没赋值」
const F NONE = LLONG_MIN;

S op(S a, S b) { return max(a, b); }

// max 的单位元：足够小的极小值 (不能写 0)
S e() { return -4e18; }

// mapping: f == NONE 说明没赋值，原样返回；
// 否则整段都变成 f，那么这段的最大值就是 f
// (注意这里不需要区间长度 —— 换成「赋值 +
// 区间和」就需要，那时 S 要存 {sum, len}，
// 返回 {f * x.len, x.len})
S mapping(F f, S x) {
    return f == NONE ? x : f;
}
```

```

// composition: 「先 g 再 f」。
// f 是真赋值 -> 它把 g 覆盖掉, 结果是 f;
// f 是 NONE (本层没动) -> 才轮到 g 生效
F composition(F f, F g) {
    return f == NONE ? g : f;
}

F id() { return NONE; } // 恒等 = 不赋值

using Seg = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

int main() {
    int n, q;
    cin >> n >> q;
    vector<S> a(n + 1, e()); // 1-based
    for (int i = 1; i <= n; i++) cin >> a[i];
    Seg seg(a);

    while (q--) {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 1) {
            ll v;
            cin >> v;
            seg.apply(l, r, v); // 闭区间
            ↪ [l, r]
        } else {
            cout << seg.prod(l, r) << "\n";
        }
    }
}

```

示例三：区间加 + 维护「最小值 & 取到最小值的权值和」。这是 DP 优化里出现频率最高的一种 S——把代价挂在 min 上、把 DP 值挂在权值和上，查询时先看区间最小代价是不是目标值（常见是 0），是才把对应的 DP 和计入答案，等价于「在所有代价最小的转移点里求 $\sum dp$ 」。

关键在于区间加不改变「谁是最小值」：整段每个元素同时 $+f$ ，最小值跟着平移，但取到最小值的那个集合不变，所以权值和原样不动——分配律因此成立，可以挂懒标记。

```

/* S = (区间最小值, 取到该最小值的位置的权值之和)
 * F = 区间加
 */
struct S {
    ll mn; // 这段区间的最小值
    ll sum; // mn 在这段里出现的那些位置的
           // 权值之和 (取模)
};

using F = ll; // 懒标记: 这段区间「待加上的数」

// op: 谁的 mn 小取谁; 一样大则权值和相加
S op(S a, S b) {
    if (a.mn < b.mn) return a;
    if (b.mn < a.mn) return b;
    ll s = a.sum + b.sum;
    if (s >= mod) s -= mod;
}

```

```

    return {a.mn, s};
}

// e: 单位元 = 空区间。
// sum 必须是 0 ——写 1 会让空区间往答案里灌值。
// mn 的哨兵有两个约束: (1) 比任何真实值都大;
// (2) 被区间加累加后仍不溢出 ——所以不能写
// LLONG_MAX / 4e18, 取 1e18 这种「够大又加得起」
// 的值 (若 mn 用 int 存, 对应取 INT32_MAX)
S e() { return {(ll) 1e18, 0}; }

// mapping: 整段 +f -> mn 平移, sum 不动。
// sum 不乘长度 (这是与「区间加 + 区间和」最大的
// 区别): 它是「取到 mn 的位置的权值和」, 区间加
// 只平移了 mn 的绝对值, 没改变谁是 mn, 也没改变
// 那些位置的权值
S mapping(F f, S x) { return {x.mn + f, x.sum};
    ↪ }

F composition(F f, F g) { return f + g; }
F id() { return 0; }

using Seg = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

```

用法骨架 (以「按代价 0 统计方案数」的线性 DP 为例):

```

Seg seg(n); // 合法位置 0..n

// 坑: 所有会被查询到的位置必须初始化成
// {0, 0} 而不是 e()。设成 e() 的话, 哨兵
// 1e18 被区间加平移后依然是天文数字, 永远
// 等不到 mn == 0, 这些位置就废了。
// {0, 0} = 「代价 0, 但还没有 dp 值」
for (int i = 1; i <= n; i++) seg.set(i, {0, 0});
seg.set(0, {0, 1}); // dp[0] = 1
↪ (空前缀)

for (int i = 1; i <= n; i++) {
    // 按题意把某些转移点的代价 +1 / -1
    // (闭区间, 不用换算)
    seg.apply(l, r, +1);

    auto q = seg.prod(0, i - 1); // 闭区间
    S cur = seg.get(i);
    if (q.mn == 0) { //
        ↪ 只有代价最小
        cur.sum += q.sum; // = 0
        ↪ 的才合法
        if (cur.sum >= mod) cur.sum -= mod;
    }
    seg.set(i, cur); // 写回 dp[i]
}

cout << seg.get(n).sum << "\n";

```

沉淀来源: HDU 2026 杭电多校第 4 场 J. Rare Game (题目链接)——那题把「同色相邻超过 3 个」的限制转成惩罚值, 用这棵树维护「惩罚为 0 的前缀里 dp 之和」。

模板本体:


```

/* ACL atcoder/lazysegtree 自包含移植 + 闭区间,
↪ 用法同上一节 segtree
* lazy_segtree(n) 合法位置 0..n (开 n+1 个叶子),
↪ 区间 [l, r] 左闭右闭
* 1-based 就只碰 1..n, 0-based 就只碰 0..n-1,
↪ 多的那格恒为 e()
* 从 vector 建: v[i] 对应位置 i, 长度自己定
* 空区间写 r == l - 1 (prod 返回 e(), apply 直接
↪ return)
*/
template<class S, S (*op)(S, S), S (*e)(),
        class F, S (*mapping)(F, S),
        F (*composition)(F, F), F (*id)()>
struct lazy_segtree {
public:
    lazy_segtree() : lazy_segtree(0) {}

    // 合法位置 0..n, 内部开 n+1 个叶子
    explicit lazy_segtree(int n)
        : lazy_segtree(vector<S>(n + 1, e())) {}

    // v[i] 对应位置 i, 合法位置 0..v.size()-1
    explicit lazy_segtree(const vector<S> &v)
        : _n(int(v.size())) {
        size = (int) bit_ceil((unsigned) _n);
        log = countr_zero((unsigned) size);
        d = vector<S>(2 * size, e());
        lz = vector<F>(size, id());
        for (int i = 0; i < _n; i++)
            d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--)
            update(i);
    }

    // a[p] = x (0 <= p <= n)
    void set(int p, S x) {
        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p
            ↪ >> i);
        d[p] = x;
        for (int i = 1; i <= log; i++) update(p
            ↪ >> i);
    }

    // 返回 a[p] (0 <= p <= n)
    S get(int p) {
        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p
            ↪ >> i);
        return d[p];
    }

    // 返回 op(a[l], ..., a[r]) 闭区间
    S prod(int l, int r) {
        r++;
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return e();
        l += size, r += size;
        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != l) push(l >>
                ↪ i);

```

```

            if (((r >> i) << i) != r)
                push((r - 1) >> i);
        }
        S sml = e(), smr = e();
        while (l < r) {
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
            l >>= 1, r >>= 1;
        }
        return op(sml, smr);
    }

    S all_prod() { return d[1]; }

    // 单点作用 a[p] = mapping(f, a[p]) (0 <= p
    ↪ <= n)
    void apply(int p, F f) {
        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p
            ↪ >> i);
        d[p] = mapping(f, d[p]);
        for (int i = 1; i <= log; i++) update(p
            ↪ >> i);
    }

    // 区间作用, 闭区间 [l, r] 内每个元素都施加 f
    void apply(int l, int r, F f) {
        r++;
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return;
        l += size, r += size;
        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != l) push(l >>
                ↪ i);
            if (((r >> i) << i) != r)
                push((r - 1) >> i);
        }
        {
            int l2 = l, r2 = r;
            while (l < r) {
                if (l & 1) all_apply(l++, f);
                if (r & 1) all_apply(--r, f);
                l >>= 1, r >>= 1;
            }
            l = l2, r = r2;
        }
        for (int i = 1; i <= log; i++) {
            if (((l >> i) << i) != l) update(l
                ↪ >> i);
            if (((r >> i) << i) != r)
                update((r - 1) >> i);
        }
    }

    // 最大的 r 使 g(op(a[l..r])) 为真 (闭区间);
    ↪ 取不到时返回 l-1
    template<class G>
    int max_right(int l, G g) {
        assert(0 <= l && l <= _n);
        assert(g(e()));
        if (l == _n) return _n - 1;
        l += size;

```

```

for (int i = log; i >= 1; i--) push(1
    ↪ >> i);
S sm = e();
do {
    while (1 % 2 == 0) 1 >>= 1;
    if (!g(op(sm, d[l]))) {
        while (1 < size) {
            push(1);
            1 = 2 * 1;
            if (g(op(sm, d[l]))) {
                sm = op(sm, d[l]);
                1++;
            }
        }
        return (1 - size) - 1;
    }
    sm = op(sm, d[l]);
    1++;
} while ((1 & -1) != 1);
return _n - 1;
}

```

// 最小的 l 使 $g(\text{op}(a[l..r]))$ 为真 (闭区间);
 ↪ 取不到时返回 $r+1$

```

template<class G>
int min_left(int r, G g) {
    r++;
    assert(0 <= r && r <= _n);
    assert(g(e()));
    if (r == 0) return 0;
    r += size;
    for (int i = log; i >= 1; i--)
        push((r - 1) >> i);
    S sm = e();
    do {
        r--;
        while (r > 1 && (r % 2)) r >>= 1;
        if (!g(op(d[r], sm))) {
            while (r < size) {
                push(r);
                r = 2 * r + 1;
                if (g(op(d[r], sm))) {
                    sm = op(d[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(d[r], sm);
    } while ((r & -r) != r);
    return 0;
}

```

private:

```

int _n, size, log;
vector<S> d;
vector<F> lz;

// 原 atcoder::internal::bit_ceil
static unsigned bit_ceil(unsigned n) {
    unsigned x = 1;
    while (x < n) x *= 2;
    return x;
}

```

```

// 原 atcoder::internal::countr_zero
static int countr_zero(unsigned n) {
    return __builtin_ctz(n);
}

void update(int k) {
    d[k] = op(d[2 * k], d[2 * k + 1]);
}

void all_apply(int k, F f) {
    d[k] = mapping(f, d[k]);
    if (k < size)
        lz[k] = composition(f, lz[k]);
}

void push(int k) {
    all_apply(2 * k, lz[k]);
    all_apply(2 * k + 1, lz[k]);
    lz[k] = id();
}
};

```

再组合下去:「区间赋值 + 区间和」把两者拼起来——S 存 {sum, len}, mapping 里 $f \neq \text{NONE}$ 时返回 $\{f * x.\text{len}, x.\text{len}\}$ 。「区间赋值 + 区间加」的复合标记要写成 `struct F { ll assign, add; }, composition` 按「先 g 再 f 」的顺序把两层压平——这类复合标记的分配律要在纸上验一遍再敲。

1.3 z_algorithm: Z 函数 (扩展 KMP)

对长度 n 的串 s , 返回数组 z , 其中 $z[i]$ 是 s 与其后缀 $s[i..n-1]$ 的最长公共前缀长度, 约定 $z[0] = n$ 。总复杂度 $O(n)$ 。

下标约定 (本节例外): 字符串这节不套本文档开头的闭区间约定——它没有 `prod` 这类区间接口, 返回的是一个和输入等长的数组。下标一律 0-based, z 的长度就是 $|s|$, 与官方 `atcoder::z_algorithm` 完全一致, 照官方文档抄调用不会差位。

接口一览:

- `z_algorithm(const string &s)`: 字符串重载, 内部逐字符转成 `vector<int>` 再跑泛型版。
- `z_algorithm(const vector<T> &s)`: 泛型版, T 只需支持 `==`。离散化后的 `vector<int>`、`vector<ll>`、`pair` 序列都能直接跑, 这是它比「只吃 `string`」的写法更值钱的地方。
- 空串返回空 `vector` (不会越界, 也不会返回 `{0}`)。

三个易错点:

- $z[0]$ 是 n 而不是 0。统计「 s 在自身出现次数」之类时若把 $z[0]$ 一并算入会多算一次; 反过来做 `exkmp` 时又必须靠它。
- 拼接法求「 a 的每个后缀与 b 的 LCP」时, 分隔符必须是两串都不会出现的字符 (小写串用 `'#'`, 读入含任意可见字符时改用 `'\1'` 这种控制符), 否则 LCP 会跨过分隔符, 答案偏大。

3. 泛型版比较的是 T 的 $==$ 。传 `vector<double>` 时浮点相等不可靠，先离散化成整数再跑。

示例一：扩展 KMP 模板题（洛谷 P5410，求 $z(b)$ 以及 a 的每个后缀与 b 的 LCP 数组 p ）。ACL 版没有单独的「两串」接口，标准做法是拼 $b + \# + a$ 跑一遍：

```
/* 题意：给 a, b, 输出
 *   z[i] = LCP(b, b[i..])      长度 |b|
 *   p[i] = LCP(b, a[i..])      长度 |a|
 * 按题目要求各输出一个带权异或和
 */
int main() {
    string a, b;
    cin >> a >> b;

    // 拼接法：分隔符要保证两串里都不出现
    // (本题只有小写字母, '#' 安全)
    vector<int> zz = z_algorithm(b + '#' + a);

    int n = a.size(), m = b.size();

    // 前 m 位就是 b 自身的 z 数组
    vector<int> z(zz.begin(), zz.begin() + m);

    // 跳过 b 和分隔符, 后 n 位就是 p 数组
    // 因为分隔符挡着, 匹配长度天然 <= m, 不用
    //   min 截断
    vector<int> p(zz.begin() + m + 1, zz.end());

    ll ans1 = 0, ans2 = 0;
    for (int i = 0; i < m; i++)
        ans1 ^= 1LL * (i + 1) * (z[i] + 1);
    for (int i = 0; i < n; i++)
        ans2 ^= 1LL * (i + 1) * (p[i] + 1);
    cout << ans1 << "\n" << ans2 << "\n";
}
```

示例二：找出模式串所有出现位置 + 最小整周期。这两件事都是 z 的直接推论，不用再写 KMP：

```
// 1) pat 在 txt 中的所有出现位置 (0-based 起点)
vector<int> find_all(const string &txt, const
    string &pat) {
    vector<int> z = z_algorithm(pat + '\1' +
        txt);
    int m = pat.size();
    vector<int> res;
    for (int i = 0; i + m <= (int) txt.size();
        i++)
        // 拼接串里 txt[i] 的下标是 m + 1 + i
        if (z[m + 1 + i] >= m) res.push_back(i);
    return res;
}

// 2) s 的最小整周期 (循环节, 整除 n)
//   z[i] + i == n 说明后缀 s[i..] 是 s 的
//   border,
//   对应周期 i; 取最小的这种 i 且 n % i == 0
int min_period(const string &s) {
    int n = s.size();
    vector<int> z = z_algorithm(s);
```

```
    for (int i = 1; i < n; i++)
        if (z[i] + i == n && n % i == 0) return
            i;
    return n; // 没有真周期, 整串自己就是周期
}
```

模板本体：

```
/* AtCoder Library v1.6 atcoder/string 的
 *   z_algorithm 自包含移植
 *   z[i] = s 与后缀 s[i..n-1] 的最长公共前缀长度,
 *   约定 z[0] = n
 *   下标 0-based, 返回长度 == 输入长度,
 *   与官方接口完全一致
 *   复杂度 O(n); 空串返回空 vector
 */
template<class T>
vector<int> z_algorithm(const vector<T> &s) {
    int n = int(s.size());
    if (n == 0) return {};
    vector<int> z(n);

    // 先当 0 参与递推 (否则 j = 0 时 z[j]
    //   会自污染),
    // 循环结束再改回 n
    z[0] = 0;

    // j = 目前「匹配段右端最靠右」的那个起点,
    // 该段是 [j, j + z[j])
    for (int i = 1, j = 0; i < n; i++) {
        int &k = z[i];

        // i 落在匹配段外 -> 从 0 暴力匹配;
        // 落在段内 -> 用对称位置 z[i-j] 当初值,
        // 但不能超出段的剩余长度 j + z[j] - i
        k = (j + z[j] <= i) ? 0 : min(j + z[j]
            - i, z[i - j]);

        // 暴力扩展 (右端不回退, 总次数线性)
        while (i + k < n && s[k] == s[i + k])
            k++;

        // 只有更靠右才换起点
        if (j + z[j] < i + z[i]) j = i;
    }

    z[0] = n;
    return z;
}

// 字符串重载: 逐字符转 int 再跑泛型版
vector<int> z_algorithm(const string &s) {
    int n = int(s.size());
    vector<int> s2(n);
    for (int i = 0; i < n; i++) s2[i] = s[i];
    return z_algorithm(s2);
}
```

和主板子那份的关系：template-main 的字符串章里有一份 lambda 写法的 Z 函数，用 $[l, r]$ 维护最右匹配区间；本节是 ACL 原版风格，用起点 j 加段长 $z[j]$ 表达同一个区间，两者等价。ACL 版多的是泛型

`vector<T>` 重载——序列而非字符串的题（如离散化后的数组求 LCP）直接能用。